

 UNIVERSITÉ ALIOUNE DIOP DE BAMBEY

# Traitement et Analyse de Données

 **PREPROCESSING** 

 **Tacko CISS**

Master 1  2025-2026  
Systèmes d'Information

- 1 Introduction au Preprocessing
- 2 Comprendre les Données
- 3 Exploration des Données
- 4 Données Aberrantes (Outliers)
- 5 Encodage des Variables
- 6 Résumé

# Qu'est-ce que le Preprocessing ?

## Définition

Le **preprocessing** (prétraitement) est l'ensemble des techniques appliquées aux données brutes pour les rendre exploitables par les algorithmes de Machine Learning.

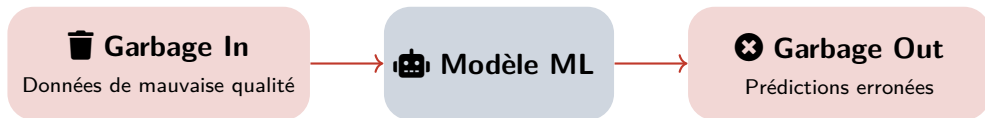
### ⚠ Problèmes courants :

- Données incomplètes
- Formats incohérents
- Valeurs aberrantes
- Variables non numériques

### ✓ Solutions :

- Nettoyage
- Transformation
- Encodage
- Normalisation

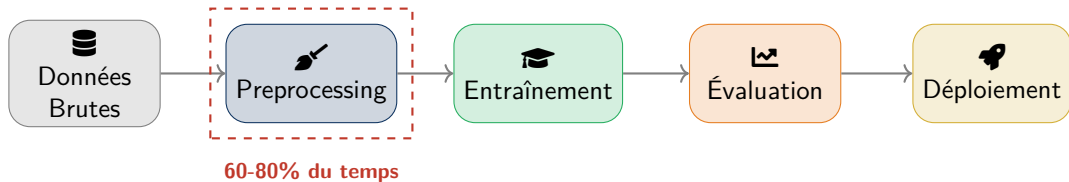
# Pourquoi le Preprocessing est crucial ?



## Statistique importante

**60-80%** du temps d'un projet Data Science est consacré au preprocessing !

# Pipeline de Machine Learning



# De l'Information à la Donnée



## Information

Fait brut non structuré  
*"Il fait chaud"*

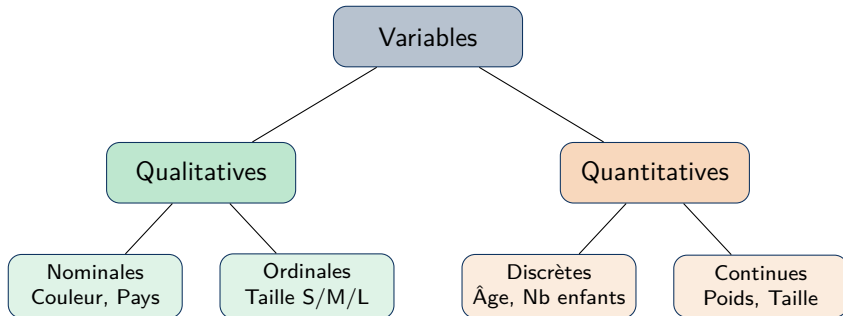
## Donnée

Fait structuré  
*Temp = 35°C*

## Indicateur

Donnée contextualisée  
*+5°C vs moyenne*

# Types de Variables



# Structures de Données en Python

## Scalaire

Une seule valeur : 42

## Vecteur (1D)

Liste de valeurs : [1, 2, 3, 4]

## Matrice (2D)

Tableau lignes  $\times$  colonnes

## DataFrame

Tableau avec noms de colonnes  
(*Pandas en Python*)

## Tenseur (nD)

Structure multidimensionnelle  
(*Images, Vidéos*)

# Première Exploration avec Pandas

## Fonctions essentielles

|                                |                                |
|--------------------------------|--------------------------------|
| <code>df.head()</code>         | Afficher les premières lignes  |
| <code>df.info()</code>         | Types et valeurs non-nulles    |
| <code>df.describe()</code>     | Statistiques descriptives      |
| <code>df.shape</code>          | Dimensions (lignes, colonnes)  |
| <code>df.isnull().sum()</code> | Compter les valeurs manquantes |

## Exemple

```
print(f"Lignes: {df.shape[0]}, Colonnes: {df.shape[1]}")
```

## Types de manquants

- **MCAR** : Aléatoire complet
- **MAR** : Aléatoire conditionnel
- **MNAR** : Non aléatoire

## Traitements possibles

- Suppression des lignes
- Imputation (moyenne, médiane)
- Imputation par modèle
- Valeur spéciale (-1, "Unknown")

## Attention

Toujours **analyser le pattern** des valeurs manquantes avant de décider !

# Qu'est-ce qu'un Outlier ?

## Définition

Une **donnée aberrante** est une observation qui s'écarte significativement des autres observations du jeu de données.

### Erreurs

Erreur de saisie  
Bug technique  
*À corriger*

### Naturelles

Événement rare  
Cas exceptionnel  
*À conserver*

### Intéressantes

Fraude  
Anomalie  
*À étudier*

## Exemple : Erreur de Saisie

| Employé | Salaire (€) | Statut      |
|---------|-------------|-------------|
| Alice   | 2 500       | Normal      |
| Bob     | 3 200       | Normal      |
| Charlie | 320 000     | ! Outlier ! |
| David   | 2 800       | Normal      |

### Analyse

Charlie gagne 320 000€ au lieu de 3 200€

→ **Erreur de saisie** (un zéro en trop)

→ **Action** : Corriger la valeur

# Méthode 1 : Règle de l'IQR

## Formule

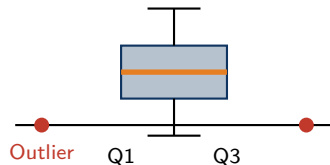
Une valeur est aberrante si elle est en dehors de :

$$[Q1 - 1.5 \times IQR ; Q3 + 1.5 \times IQR]$$

où  $IQR = Q3 - Q1$  (écart interquartile)

## Exemple numérique

- $Q1 = 13$ ,  $Q3 = 21$
- $IQR = 21 - 13 = 8$
- Bornes :  $[1; 33]$
- Valeur 85 → **Outlier !**



## Méthode 2 : Z-Score

### Formule

$$Z = \frac{x - \mu}{\sigma}$$

Si  $|Z| > 3$  alors c'est un **Outlier**

### Interprétation

Le Z-score mesure combien d'écart-types séparent une valeur de la moyenne.

- $|Z| < 1$  : Valeur proche de la moyenne
- $1 < |Z| < 2$  : Valeur modérément éloignée
- $2 < |Z| < 3$  : Valeur éloignée
- $|Z| > 3$  : **Outlier probable**

| Stratégie            | Quand ?           | Méthode              |
|----------------------|-------------------|----------------------|
| <b>Suppression</b>   | Erreurs évidentes | Filtrer le DataFrame |
| <b>Winsorisation</b> | Limiter l'impact  | Capper aux bornes    |
| <b>Log transform</b> | Asymétrie forte   | Transformation log   |
| <b>Imputation</b>    | Remplacer         | Médiane ou moyenne   |

## Règle d'or

 Ne jamais supprimer automatiquement tous les outliers !  
Toujours **analyser** et **comprendre** avant de décider.

# Pourquoi Encoder ?

## Problème

Les algorithmes de Machine Learning ne comprennent que les **nombres** !



## Solution

Transformer les variables catégorielles en valeurs numériques via l'**encodage**.

# Label Encoding

## Principe

Attribuer un **entier unique** à chaque catégorie.

| Original | Encodé |
|----------|--------|
| Bac      | 0      |
| Licence  | 1      |
| Master   | 2      |
| Doctorat | 3      |

Adapté pour

Variables **ordinales**  
(avec ordre naturel)

## Code Python

```
from sklearn.preprocessing import  
LabelEncoder  
  
le = LabelEncoder()  
df['col_enc'] =  
le.fit_transform(df['col'])
```

# Attention au Label Encoding !

## Danger avec les variables nominales

Si on encode les couleurs : Rouge=0, Vert=1, Bleu=2

L'algorithme peut interpréter :

- $Bleu > Vert > Rouge$  ✗
- $Rouge + Vert = Bleu$  ✗

Ce qui n'a **aucun sens** !

→ **Solution** : Utiliser **One-Hot Encoding** pour les variables nominales

# One-Hot Encoding

## Principe

Créer une **colonne binaire** (0/1) pour chaque catégorie.

**Avant :**

| Couleur |
|---------|
| Rouge   |
| Bleu    |
| Vert    |
| Rouge   |

**Après :**

| Rouge | Bleu | Vert |
|-------|------|------|
| 1     | 0    | 0    |
| 0     | 1    | 0    |
| 0     | 0    | 1    |
| 1     | 0    | 0    |

## Code Python

```
df_encoded = pd.get_dummies(df, columns=['couleur'])
```

# Comparaison des Méthodes d'Encodage

| Critère          | Label Encoding | One-Hot       |
|------------------|----------------|---------------|
| Type de variable | Ordinale       | Nominale      |
| Nb de colonnes   | 1              | n catégories  |
| Ordre artificiel | Oui ✗          | Non ✓         |
| Mémoire          | Faible ✓       | Élevée ✗      |
| Exemple          | Taille S/M/L   | Couleur, Pays |

## Règle simple

- Variable avec **ordre naturel**  $\Rightarrow$  **Label Encoding**
- Variable **sans ordre**  $\Rightarrow$  **One-Hot Encoding**

# Les 5 Piliers du Preprocessing



**Valeurs  
Manquantes**



**Nettoyage**



**Trans-  
formation**



**Encodage**



**Enrichis-  
sment**

- ① **Le preprocessing est crucial** : 60-80% du temps d'un projet ML
- ② **Garbage in, garbage out** : Qualité données = Qualité modèle
- ③ **Types de variables** : Qualitatives vs Quantitatives
- ④ **Explorer avant d'agir** : Toujours visualiser et décrire
- ⑤ **Outliers** : Analyser avant de supprimer (IQR, Z-score)
- ⑥ **Encodage** : Label (ordinal) vs One-Hot (nominal)
- ⑦ **Documenter** : Garder une trace des décisions

## Exploration :

- ✓ Structure des données
- ✓ Types de variables
- ✓ Valeurs manquantes
- ✓ Distributions
- ✓ Outliers

## Traitement :

- ✓ Imputation des manquants
- ✓ Gestion des outliers
- ✓ Encodage des catégories
- ✓ Normalisation/Scaling
- ✓ Feature engineering

# Merci pour votre attention !

 Questions ?

 Tacko CISSMaster 1 Systèmes d'Information

 Université Alioune Diop de Bambey – 2025-2026